Java OOP — Part 3.3: Inheritance and Polymorphism

Part 3.3 covers the relationships between classes: how one class **reuses and specializes** another through inheritance, how the same call can behave differently depending on the object (polymorphism), when to prefer composition, and how copying objects works. It builds directly on constructors (3.1.5), initialization order (3.1.6) and the keywords in 3.2.

3.3.1 Inheritance: extends, IS-A and What a Subclass Gets

Inheritance lets a class acquire the fields and methods of another class and then **add to them or change them**. The class being inherited from is the **superclass** (also called parent or base class). The class that inherits is the **subclass** (child or derived class).

```
class Employee {                                    // superclass
    protected String name;
    protected double baseSalary;

    Employee(String name, double baseSalary) { this.name = name;
this.baseSalary = baseSalary; }
    double pay()        { return baseSalary; }
    String describe()   { return name + " earns " + pay(); }
}

class Manager extends Employee {                    // subclass: extends = "is
a kind of"
    private final double bonus;                     // new state

    Manager(String name, double baseSalary, double bonus) {
        super(name, baseSalary);                    // run the parent
constructor first
        this.bonus = bonus;
    }
    @Override double pay() { return super.pay() + bonus; }      //
specialized behavior
    void approveLeave(Employee e) { /* new behavior only managers have */ }
}
```

```
Manager m = new Manager("Asha", 80_000, 20_000);
System.out.println(m.describe());    // Asha earns 100000.0   describe() is
inherited; pay() is the Manager's version
```

`Manager` did not write `name`, `baseSalary` or `describe()`, yet it has them. It added one field and one method, and changed one method. That is the three things a subclass does: **reuse**, **extend**, **specialize**.

The IS-A test

Inheritance is for **IS-A** relationships: a `Manager` **is an** `Employee`, a `Dog` **is an** `Animal`, an `ArrayList` **is a** `List`. Before writing `extends`, say the sentence out loud. If it sounds wrong ("a `Stack` is a `Vector`", which the JDK's own `java.util.Stack` claims, and regrets), you probably want composition (3.3.2 and 3.3.6).

IS-A also means **substitutability**: anywhere the code expects an `Employee`, a `Manager` must work correctly. That is the Liskov Substitution Principle (3.3.6) and the basis of polymorphism.

```
Employee e = new Manager("Asha", 80_000, 20_000);   // a Manager can be used
as an Employee
System.out.println(e.pay());                         // 100000.0: still the
Manager's pay() at runtime
// e.approveLeave(x);                                // ERROR: the reference
type Employee has no such method
```

Rules for `extends`

- A class extends **exactly one** class (single inheritance). Every class that names no parent extends `Object`.
- You cannot extend a **`final`** class, or a class whose **constructors are all inaccessible** (`private`), or a **`sealed`** class that does not list you in `permits`.
- Inheritance cannot be **circular** (`A extends B`, `B extends A` is an error).
- A **subclass constructor must call a parent constructor**, explicitly with `super(...)` or implicitly with `super()`. The parent must therefore offer an accessible constructor that fits (3.1.5).

- An **interface** is implemented with `implements`; an interface may `extends` other interfaces (3.4). A class cannot extend an interface and an interface cannot extend a class.
- The subclass can live in a **different package**; then only `public` and `protected` parent members are reachable.

What is inherited and what is not

Parent member	Inherited by the subclass?	Notes
`public` and `protected` fields and methods	**Yes**	Usable directly by name
Package-private members	Only if the subclass is in the **same package**	Otherwise not visible
`private` members	**No** (not accessible), but they **exist in every object**	Use parent's public or protected methods to reach them
Constructors	**No**	Called with `super(...)`; each class defines its own
Static members	Accessible through the subclass name, but a same-signature static method **hides** (does not override)	See 3.2.1
Instance and static initializer blocks	Not inherited as members	They run as part of the parent's construction and class initialization
Nested classes	Accessible if visible (not `private`)	
Interface `default` methods	Yes, through the implemented interface	See 3.4

"Not inherited" does not mean "not there." A parent's `private` field is still stored in the child object's memory. A `Manager` object contains `name`, `baseSalary` **and** `bonus`; code in `Manager` just cannot name a private parent field directly.

```
class Account {
    private double balance;                          // private: stored in
every Account and every subclass object
    public double getBalance()  { return balance; }  // the door the subclass
may use
    protected void credit(double amt) { balance += amt; }
}
class SavingsAccount extends Account {
    void addInterest(double rate) {
        // balance += balance * rate;                // ERROR: balance is
private in Account
        credit(getBalance() * rate);                 // OK: through the
parent's non-private methods
    }
}
```

How the object looks in memory

A subclass object is laid out with the **parent's fields first, then the child's**. One object, one allocation, a single identity.

```
Manager object on the heap
+-----------------------------+
| header (class pointer, ...) |
| name          (from Employee)|   <- the Employee part: a Manager object CAN
be viewed as an Employee
| baseSalary    (from Employee)|
| bonus         (from Manager) |   <- the Manager part
+-----------------------------+
```

Because the parent part sits at the front with the same layout, code written for `Employee` can read a `Manager` object unchanged. That is why **upcasting costs nothing** at runtime.

Constructors and `super` (quick recap)

- Constructors run **parent first**: `new Manager(...)` runs `Object()`, then `Employee(...)`, then the rest of `Manager(...)`.
- If you do not write `super(...)`, the compiler inserts `super()`. If the parent has no no-arg constructor, you must call a parent constructor explicitly.
- `super.method()` calls the parent's version of an overridden method, and `super.field` reads a hidden parent field (3.1.5).

Benefits and costs of inheritance

Benefits	Costs
Reuse: common code lives once in the parent	**Tight coupling:** the child depends on the parent's implementation details
Extensibility: new subclasses add behavior without changing old code	**Fragile base class:** parent changes can break children (3.3.6)
Polymorphism: treat different subclasses uniformly	**Breaks encapsulation** if children rely on protected internals
Specialization: override only what differs	**Deep hierarchies** become hard to read and change

Guideline: use inheritance for a genuine IS-A relationship where the subclass can stand in for the parent everywhere. For "uses" and "has" relationships, use composition.

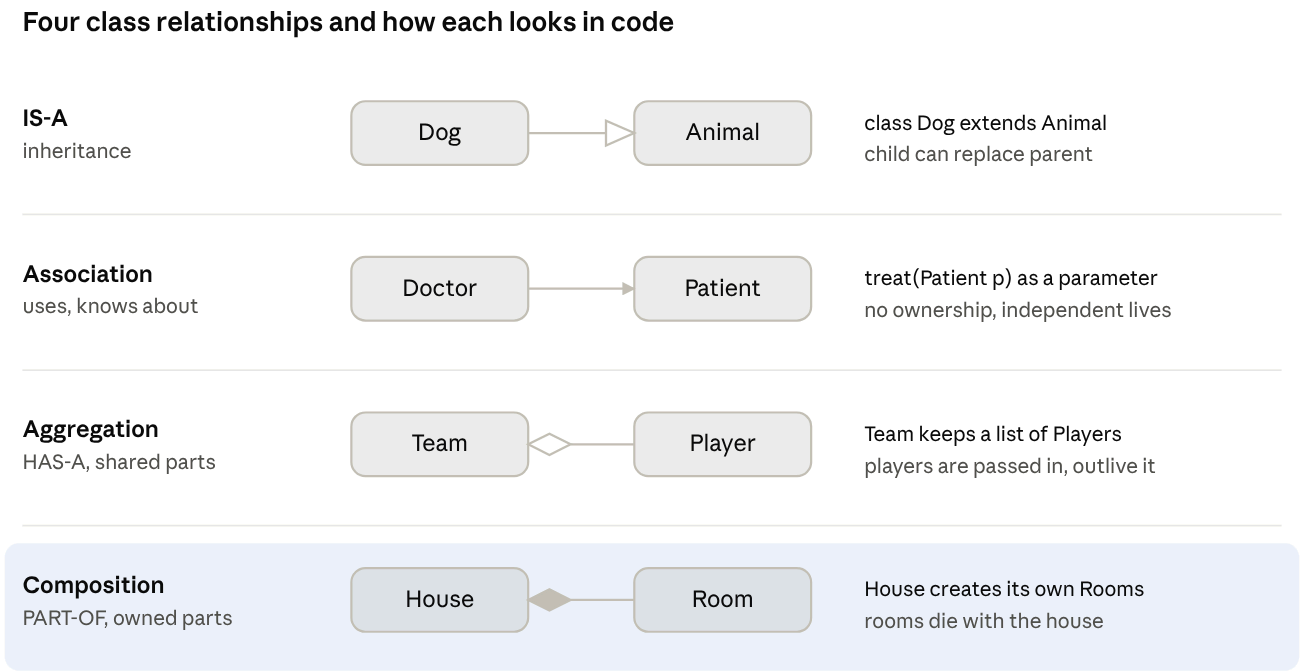
Interview traps

- *Can a subclass access the parent's private members?* Not directly; through the parent's non-private methods or via nested-class access.
- *Does a subclass object contain the parent's private fields?* Yes; they are in the object, just not accessible by name.
- *Are constructors inherited?* No. That is why a subclass must call `super(...)`.
- *Can a class extend two classes?* No. Use interfaces for multiple types (3.3.3).
- *Can a class extend itself, or two classes extend each other?* No; circular inheritance is a compile error.
- *Is `static` inherited?* Static members can be accessed through the subclass, but they are not overridden, only hidden.
- *What is the parent of a class that has no `extends` clause?* `java.lang.Object`.

3.3.2 Relationships Between Classes: IS-A, HAS-A and PART-OF

Classes rarely work alone. Three questions tell you which relationship to model:

Question	Relationship	Java expression
Is X **a kind of** Y?	**IS-A**: inheritance	`class X extends Y`, `class X implements Y`
Does X **have** a Y as a part or collaborator?	**HAS-A**: aggregation or composition	A field of type `Y` inside `X`
Does X merely **use** a Y for a moment?	**Association / dependency**	A method parameter, local variable or return type



Aggregation and composition are both HAS-A. The difference is lifetime: aggregated parts outlive the whole, composed parts are created, owned and destroyed with it.

IS-A, association, aggregation and composition

The drawing uses UML notation: a **hollow triangle** points to the parent in inheritance, a **plain arrow** is a simple association, a **hollow diamond** sits on the owner's side in aggregation, and a **filled diamond** in composition. Each row shows what the relationship looks like in Java.

IS-A: inheritance

Covered in 3.3.1. Use it only when the child can fully **substitute** for the parent. An `ElectricCar` IS-A `Car`; an `ArrayList` IS-A `List`.

Association

An **association** is a general relationship in which one object knows about or uses another, **without owning it**. Both live independently.

```
class Doctor {
    void treat(Patient p) { /* uses p for the duration of the call */ }   //
dependency: parameter
}

class Student {
    private final List<Course> courses = new ArrayList<>();              //
association: Student knows its Courses
    void enroll(Course c) { courses.add(c); }
}
```

Associations vary in **multiplicity** (one-to-one, one-to-many, many-to-many) and **direction** (`Student` knows `Course`, or both know each other). A short-lived use through a parameter or local variable is often called a **dependency**, the weakest form.

Aggregation: HAS-A with shared, independent parts

Aggregation is a HAS-A relationship in which the parts **can exist without the whole** and may be shared. The whole does not create or destroy them. If the whole is discarded, the parts live on.

```
class Player { final String name; Player(String name) { this.name = name; } }

class Team {
    private final List<Player> players;
    Team(List<Player> players) { this.players = new ArrayList<>(players); }
// parts are created OUTSIDE and passed in
}

Player p1 = new Player("Asha");
Team a = new Team(List.of(p1));
Team b = new Team(List.of(p1));   // the same Player can belong to more than
one Team
// when 'a' is garbage collected, p1 still exists
```

Other examples: a `Department` and its `Professors`, a `Library` and its `Books`, a `Playlist` and its `Songs`.

Composition: PART-OF with owned parts

Composition is a stronger HAS-A relationship: the whole **owns** its parts, **creates** them, and they have **no meaning or life outside it**. When the whole is destroyed, the parts are destroyed with it. A part belongs to exactly one whole.

```
class Engine {
    private final int horsepower;
    Engine(int hp) { this.horsepower = hp; }
}

class Car {
    private final Engine engine;                        // created, owned and
hidden by Car
    Car(int hp) { this.engine = new Engine(hp); }      // Car creates the
part itself
    // no getEngine() that hands the object to others, no setEngine(...)
}
```

Other examples: `House` and `Room`, `Order` and `OrderLine`, `Invoice` and its line items. Java has no keyword for composition. You express it by: the whole **creating** the parts (not receiving them), **not exposing** the parts for reuse elsewhere, and letting the parts become unreachable (garbage) together with the whole. A part class is often a **private nested class** of the whole.

In JPA / Hibernate, the same idea appears as `@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)` for `Order` to `OrderLine` (children are saved and deleted with the parent), while a plain `@ManyToOne` reference to a `Customer` is an association or aggregation.

Telling the three apart

Question	Association	Aggregation	Composition
Does one own the other?	No	Weakly ("has")	**Strongly ("part of")**
Who creates the part?	Anyone	Usually outside, then passed in	**The whole**

Question	Association	Aggregation	Composition
Can a part exist without the whole?	Yes	Yes	**No**
Can a part be shared by several wholes?	Yes	Yes	**No**
Lifetime link	None	None	**Same lifetime**
Typical Java form	Parameter, local variable, simple field	Field filled from a constructor argument or setter	Field created by the whole in its constructor

A quick test: **"If I delete the whole, should the part disappear too?"** Yes means composition; no means aggregation or association.

HAS-A or IS-A?

A common design mistake is to inherit just to reuse code:

```
class Stack<E> extends ArrayList<E> { }          // WRONG: a Stack is not an
ArrayList; callers can now call add(0, x), remove(5)...

class Stack<E> {                                 // RIGHT: HAS-A list,
exposes only stack operations
    private final Deque<E> items = new ArrayDeque<>();
    void push(E e) { items.push(e); }
    E pop()        { return items.pop(); }
}
```

Composition buys you control over exactly what you expose (encapsulation), the freedom to **change the inner part** later, the ability to **swap the part at runtime** when it is typed by an interface (the Strategy pattern), and protection from the fragile base class problem (3.3.6). Prefer composition unless there is a true IS-A relationship.

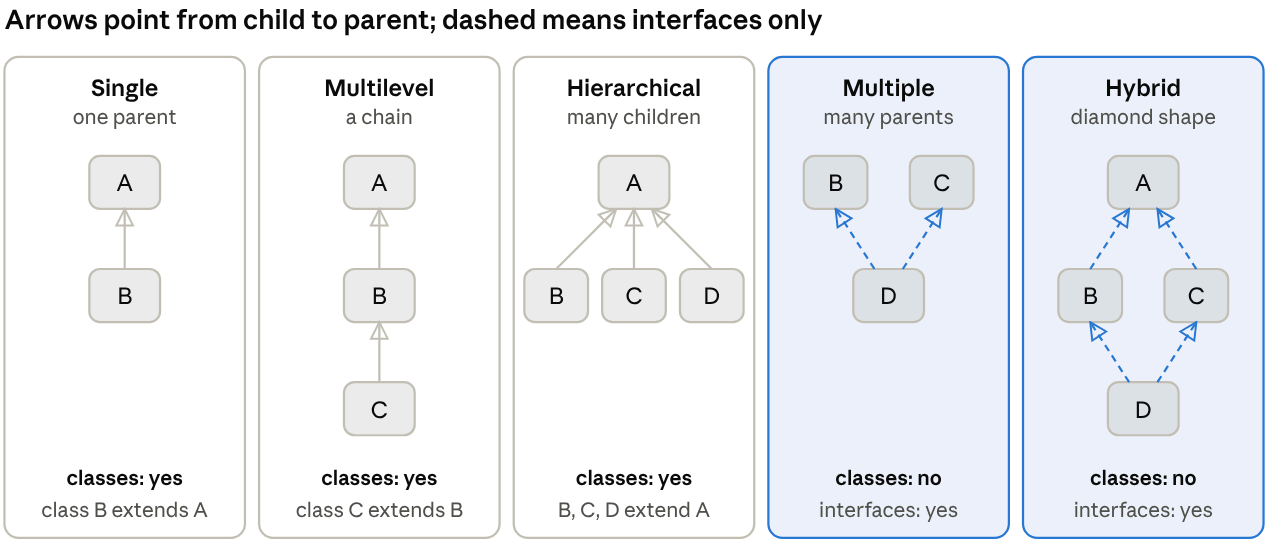
Interview traps

- *Difference between aggregation and composition?* Lifetime and ownership: aggregated parts live independently and may be shared; composed parts are owned by one whole and die with it.

- *Association vs aggregation?* Association is any "uses" link. Aggregation is a specific HAS-A association in which one object is the whole and the other its part.
- *Does Java have special syntax for composition?* No; it is a design relationship expressed through ownership and creation.
- *Which is more tightly coupled: inheritance or composition?* Inheritance, because the child depends on the parent's implementation details.
- *Give an example of HAS-A.* `Car` has an `Engine`; `Employee` has an `Address`.

3.3.3 Types of Inheritance

Textbooks name five shapes of inheritance, depending on how many parents and children classes have.



The five types of inheritance and Java's support for each

Type	Shape	Java with **classes**	Java with **interfaces**
Single	One child, one parent	Yes	Yes
Multilevel	A chain: C extends B extends A	Yes	Yes
Hierarchical	One parent, several children	Yes	Yes
Multiple	One child, several parents	**No**	**Yes**

Type	Shape	Java with **classes**	Java with **interfaces**
Hybrid	A mix of the above, such as the diamond	**No** (if it includes multiple)	**Yes**

Single inheritance

One class extends one class.

```
class Vehicle { void start() { System.out.println("starting"); } }
class Car extends Vehicle { void honk() { System.out.println("beep"); } }

Car c = new Car();
c.start();     // inherited
c.honk();      // its own
```

Multilevel inheritance

A class extends a class that itself extends another, forming a **chain**. A member is inherited through every level.

```
class Animal   { void eat()   { System.out.println("eating"); } }
class Mammal extends Animal { void breathe() {
System.out.println("breathing"); } }
class Dog    extends Mammal { void bark()    { System.out.println("woof"); }
}

new Dog().eat();       // from Animal, two levels up
```

Every Java class is already in a chain that ends at `Object`: `Dog` to `Mammal` to `Animal` to `Object`. Constructors run from the **top of the chain downward** (3.1.5). Keep chains short (about three levels of your own classes): each level adds coupling and makes the behavior harder to trace.

Hierarchical inheritance

Several classes extend **the same parent**, each specializing it differently.

```
abstract class Shape { abstract double area(); }
class Circle    extends Shape { double r;     double area() { return Math.PI
* r * r; } }
class Rectangle extends Shape { double w, h;  double area() { return w * h; }
}
class Triangle  extends Shape { double b, h;  double area() { return 0.5 * b
* h; } }
```

This is the most common shape in practice and the base of polymorphism: a `List<Shape>` can hold all three. The JDK's `Number` has `Integer`, `Long`, `Double` and others as children; `Throwable` has `Exception` and `Error`.

Multiple inheritance

Multiple inheritance means one class inherits from **several parent classes**. Java deliberately **does not allow it for classes**:

```
class A { }
class B { }
// class C extends A, B { }       // COMPILE ERROR: a class can extend only
one class
```

Why: the diamond problem. Suppose `B` and `C` both extend `A` and both override `A.greet()`, and `D` extends both `B` and `C`. When `D` calls `greet()`, which version runs: `B`'s or `C`'s? And does `D` get **one** copy of `A`'s fields or **two**? Languages that allow it (C++) need extra rules such as virtual base classes and still confuse programmers. Java removes the problem by allowing a class **one parent class only**, so state and constructors always have a single, clear source.

```
        A { greet() }
       /             \
  B { greet() }   C { greet() }
       \             /
        D extends B, C     <- which greet()? which copy of A's fields?
```

Java's answer: interfaces. A class may **implement many interfaces**, and an interface may **extend many interfaces**. This is multiple inheritance of **type and contract**, not of state, so there is no field-duplication problem.

```
interface Flyer   { void fly(); }
interface Swimmer { void swim(); }
interface Walker  { default void walk() { System.out.println("walking"); } }

class Duck implements Flyer, Swimmer, Walker {          // three parents, one
class
    public void fly()  { System.out.println("flap"); }
    public void swim() { System.out.println("paddle"); }
}
interface Amphibious extends Flyer, Swimmer { }         // an interface may
extend several interfaces
```

Since Java 8, `default` methods let interfaces carry behavior, so the diamond can appear again. Java resolves it with **explicit rules** (full treatment in 3.4):

1. **A class method wins** over any interface default ("class wins").
2. Otherwise the **more specific interface** wins (the one that extends the other).
3. If still ambiguous, the compiler **forces you to override** and choose.

```
interface A { default String hello() { return "A"; } }
interface B { default String hello() { return "B"; } }
class C implements A, B {
    @Override public String hello() { return A.super.hello() +
B.super.hello(); }   // must resolve it yourself
}
```

Hybrid inheritance

Hybrid inheritance combines two or more of the types above, typically including multiple inheritance (the diamond). With classes alone Java cannot express it; with **interfaces** it can:

```
interface Payable   { double pay(); }
interface Reportable{ String report(); }
interface Auditable extends Payable, Reportable { }     // multiple interface
inheritance

abstract class Staff { protected String name; }                        //
class hierarchy
class Manager extends Staff implements Auditable {                    // one
class parent + interface parents
    public double pay()    { return 100_000; }
    public String report() { return name + " report"; }
}
```

Real examples from the JDK

- **Multilevel:** `ArrayList` extends `AbstractList`, which extends `AbstractCollection`, which extends `Object`.
- **Hierarchical:** `Number` is extended by `Integer`, `Long`, `Double`, `BigDecimal` and more.
- **Multiple types through interfaces:** `ArrayList` implements `List`, `RandomAccess`, `Cloneable` and `Serializable`.

Guidance

- Prefer **shallow hierarchies** and **composition** (3.3.2, 3.3.6) over deep chains.
- Use **interfaces** to model capabilities (`Comparable`, `Runnable`) that cut across class families.
- Use **sealed** hierarchies (3.5) when the set of subclasses must be closed.

Interview traps

- *Does Java support multiple inheritance?* Not of classes; yes of interfaces (type), and since Java 8 of default method behavior, with explicit conflict rules.
- *Why no multiple inheritance of classes?* The diamond problem: ambiguity of inherited methods and duplicated state.
- *Can an interface extend multiple interfaces?* Yes. *Can an interface extend a class?* No.
- *Is `Object` part of every multilevel chain?* Yes, always at the top.

- *Can a class have more than one subclass?* Yes; that is hierarchical inheritance. There is no limit.

3.3.4 Method Overloading vs Overriding

Two features share the idea "same method name, different behavior" but work in completely different ways and at different times.

	Overloading	Overriding
Meaning	Several methods with the **same name and different parameters**	A subclass **replaces** an inherited method with the **same signature**
Where	Same class, or class and its subclasses	Parent and child (needs inheritance)
Signature	Parameter list **must differ** (type, number or order)	Parameter types **must match** exactly
Return type	Can differ freely (but cannot be the only difference)	Same, or a **subtype** (covariant)
Access modifier	Can differ	Same or **wider**
Checked exceptions	Can differ	Same, **narrower**, or none
Which version runs	Chosen by the **compiler**, from the **declared types** of the arguments	Chosen at **runtime**, from the **actual class** of the object
Also called	Compile-time (static) polymorphism, early binding	Runtime (dynamic) polymorphism, late binding
`static`, `private`, `final` methods	Can be overloaded	Cannot be overridden

Overloading

Define several methods with the **same name** but **different parameter lists**. The **signature** is the name plus the parameter types in order; the return type is not part of it.

```
class Printer {
    void print(int n)           { System.out.println("int: " + n); }
    void print(double d)        { System.out.println("double: " + d); }
    void print(String s)        { System.out.println("String: " + s); }
    void print(String s, int n)  { System.out.println(s + " x " + n); }
// different number
    void print(int n, String s)  { System.out.println(n + " " + s); }
// different order
    // int print(int n) { return n; }    // ERROR: same signature as
print(int); return type does not count
}
```

What may differ: the number, types or order of parameters. What does **not** make an overload: the return type, parameter **names**, `throws` clauses, or access modifiers on their own. Constructors can be overloaded in the same way (3.1.5).

How the compiler picks a method. For a call, the compiler looks only at the **compile-time types of the arguments** and chooses in **three phases**, stopping at the first phase that finds a match:

1. **No conversion except subtyping and primitive widening.** (`int` to `long` , `String` to `Object` .)
2. **Boxing or unboxing allowed.** (`int` to `Integer` .)
3. **Varargs allowed.**

If several methods match in the same phase, the **most specific** one wins; if none is more specific than the others, it is an **ambiguity error**.

```
static void f(long x)    { System.out.println("long"); }
static void f(Integer x) { System.out.println("Integer"); }
static void f(int... x)  { System.out.println("varargs"); }
f(5);                     // long      phase 1: widening int to long beats
boxing to Integer

static void g(Object o)  { System.out.println("Object"); }
static void g(String s)  { System.out.println("String"); }
g("hi");                  // String    most specific applicable
g(null);                  // String    String is more specific than Object

static void h(Integer i) { }
static void h(String s)  { }
// h(null);               // COMPILE ERROR: ambiguous, neither is more
specific

char c = 'a';
System.out.println(c);    // a    println(char)
System.out.println(c + 1);// 98   int arithmetic, println(int)
```

The compile-time type decides, not the runtime type.

```
static void show(Object o) { System.out.println("Object"); }
static void show(String s) { System.out.println("String"); }

Object x = "hello";        // runtime type String, compile-time type Object
show(x);                   // Object   overload is chosen BEFORE the program
runs
show((String) x);          // String
```

More overloading rules

- Two methods whose parameter types are the same **after generic erasure** clash: `void f(List<String> l)` and `void f(List<Integer> l)` is a compile error (both erase to `List`).
- Overloaded methods can be inherited and overloaded again in a subclass; the subclass then has **all** of them.
- Prefer a **different name** when overloads would be confusing (`removeAt(int index)` vs `remove(Object o)`; the JDK's `List.remove(int)` and `remove(Object)` famously trap `List<Integer>` users).

- Varargs overloads are the last choice and can be ambiguous with others; avoid mixing them.

Overriding

Overriding is when a subclass provides its own implementation of an **inherited instance method**, keeping the **same signature**. It is what makes polymorphism work.

```
class Animal {
    String sound() { return "..."; }
    Animal reproduce() { return new Animal(); }
}
class Dog extends Animal {
    @Override String sound() { return "woof"; }                  // same
signature: overrides
    @Override Dog reproduce() { return new Dog(); }              // covariant
return type: Dog is an Animal
}

Animal a = new Dog();
System.out.println(a.sound());     // woof   chosen at RUNTIME by the real
object
```

Rules for a valid override

Rule	Detail
Method name and **parameter types**	Identical
Return type	Same, or a **subtype** for reference types (**covariant return**). Primitives and `void` must match exactly
Access	Same or **wider**: `protected` may become `public`; `public` cannot become `protected`
Checked exceptions	May throw the same, **narrower** (subclass) checked exceptions, or none; may **not** add new or broader checked ones. Unchecked exceptions are unrestricted
Method kind	Only **instance** methods that are not `private`, `final` or `static`; constructors cannot be overridden

Rule	Detail
Generics	The signature must match after substituting type arguments
`abstract`	An abstract method must be overridden by a concrete subclass

```
class Reader1 { void load() throws IOException { } }
class FileLoader extends Reader1 {
    @Override void load() throws FileNotFoundException { }   // OK: narrower
(a subclass of IOException)
}
class Bad extends Reader1 {
    // @Override void load() throws Exception { }            // ERROR:
Exception is broader than IOException
}
```

The exception rule exists because a caller coded against the parent type is prepared only for the parent's checked exceptions.

Always write `@Override`. The annotation makes the compiler verify that you really override something. Without it a typo or wrong parameter type silently creates a **new overload** instead:

```
class Point {
    int x, y;
    public boolean equals(Point p) { return x == p.x && y == p.y; }    //
OVERLOAD, not override! parameter is Point, not Object
}
Set<Point> set = new HashSet<>();
set.add(new Point());
System.out.println(set.contains(new Point()));   // false: HashSet calls
equals(Object), which is still the identity version

// With @Override the compiler would have rejected it:
// @Override public boolean equals(Point p) { ... }    // ERROR: method does
not override a method from its superclass
```

Covariant return types and bridge methods. `Dog reproduce()` overriding `Animal reproduce()` is legal. At the bytecode level the JVM matches methods by exact descriptor, so `javac` generates a hidden **bridge method** `Animal reproduce()` in `Dog`

that calls the real `Dog reproduce()`. You never see it, except in stack traces and reflection.

Calling the parent's version. `super.method()` runs the parent's implementation, usually to extend rather than replace it:

```
@Override String describe() { return super.describe() + " (managed)"; }
```

What cannot be overridden, and what happens instead

Member	Result of declaring the same signature in a subclass
`final` method	Compile error
`static` method	**Hides** the parent's; bound at compile time (3.2.1)
`private` method	A brand new, unrelated method; the parent's is not inherited
Constructor	Not inherited; no overriding
Field	**Hides** the parent's field; fields are never polymorphic (3.3.5)

Overloading and overriding together

This classic puzzle tests both mechanisms:

```
class A {
    void f(Object o) { System.out.println("A.f(Object)"); }
}
class B extends A {
    @Override void f(Object o) { System.out.println("B.f(Object)"); }     //
overrides
    void f(String s)           { System.out.println("B.f(String)"); }      //
overloads (new method)
}

A a = new B();
a.f("x");      // B.f(Object)
```

Walk through it: **at compile time** the reference type is `A`, and `A` has only `f(Object)`, so that signature is selected. **At runtime** the real object is a `B`, which overrides `f(Object)`, so `B.f(Object)` runs. The extra `B.f(String)` is invisible through an `A` reference. With `B b = new B(); b.f("x")` the compiler would pick `f(String)`.

Rule of thumb: overload resolution looks at the **reference and argument types** (compile time); overriding selects the **object's class** (runtime).

Interview traps

- *Can we overload by return type alone?* No; the signature excludes the return type.
- *Can we overload `main`?* Yes, but the JVM launches only `public static void main(String[])`.
- *Can we override a static or private method?* No. Static methods are hidden; private methods are not inherited.
- *Can an overriding method be less accessible?* No, only the same or wider.
- *Can an overriding method throw a broader checked exception?* No, only narrower or none.
- *Is overloading polymorphism?* Yes, compile-time polymorphism; overriding is runtime polymorphism.
- *What is a covariant return type?* An overriding method that returns a subtype of the parent method's return type.
- *What is `@Override` for?* A compile-time check that the method really overrides; it prevents accidental overloads.

3.3.5 Polymorphism, Dynamic Dispatch and Hiding

Polymorphism means "many forms": **one call, many possible behaviors**, depending on the object it is applied to. It lets you write code against a general type and have it work correctly for every specific type, including ones that do not exist yet.

Kind	Mechanism	Decided	Example
Compile-time (static)	Method **overloading**, operators	By the compiler, from declared types	`print(int)` vs `print(String)`; `+` for numbers and strings

Kind	Mechanism	Decided	Example
Runtime (dynamic, subtype)	Method **overriding** through inheritance or interfaces	By the JVM, from the object's real class	`shape.area()`
Parametric	**Generics**	At compile time, with erasure	`List<T>` works for any `T`

When people say just "polymorphism" in an OOP interview, they mean the **runtime** kind.

Runtime polymorphism in action

```
abstract class Payment {
    abstract void pay(double amount);
}
class UpiPayment extends Payment  { void pay(double a) {
System.out.println("UPI " + a); } }
class CardPayment extends Payment { void pay(double a) {
System.out.println("Card " + a); } }
class WalletPayment extends Payment { void pay(double a) {
System.out.println("Wallet " + a); } }

class Checkout {
    void process(Payment p, double amount) {     // written once, against the
parent type
        p.pay(amount);                           // which pay()? decided at
runtime
    }
}

Checkout c = new Checkout();
c.process(new UpiPayment(), 250);       // UPI 250.0
c.process(new CardPayment(), 900);      // Card 900.0
```

`Checkout` never mentions UPI or cards. Adding `NetBankingPayment` needs **no change** to `Checkout`. This is the **Open/Closed Principle**: open for extension, closed for modification. Without polymorphism you would write `if (p instanceof UpiPayment) ... else if ...` in every place that handles payments, and edit all of them for each new type.

The ingredients: (1) inheritance or interface implementation, (2) **overriding**, (3) an **upcast** (a parent-typed reference holding a child object).

Upcasting and downcasting

```
Payment p = new UpiPayment();          // upcast: implicit, always safe
p.pay(100);                            // OK: pay() exists on Payment
// p.verifyVpa();                      // ERROR: the reference type decides
which members you may call

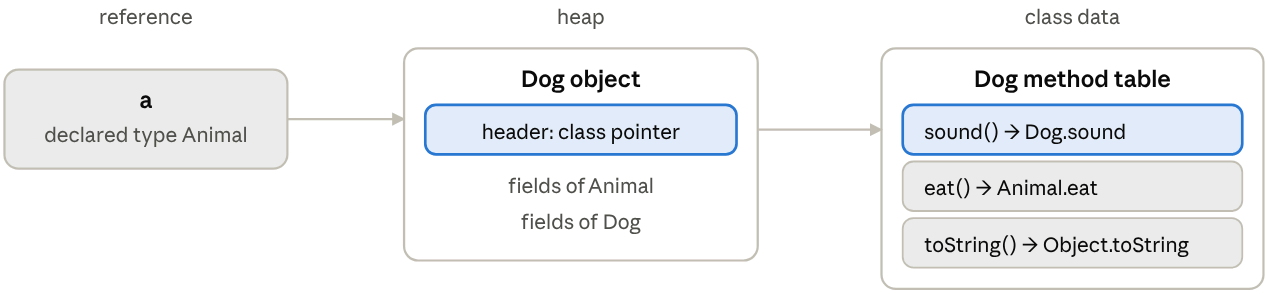
if (p instanceof UpiPayment u) {       // test, cast and bind (Java 16+)
    u.verifyVpa();                     // child-specific method through a
child-typed reference
}
CardPayment bad = (CardPayment) p;     // compiles; throws ClassCastException
at runtime
```

Two types are always at play: the **reference (static) type** controls what the **compiler** allows you to call; the **object (dynamic) type** controls which **implementation** actually runs.

How dynamic dispatch works

The JVM implements overriding with a **virtual method table** (vtable) in each class's data. Each non-private, non-static, non-final instance method has a **slot**. A subclass copies its parent's table and **replaces the slots it overrides**. Every object's header holds a **class pointer** to its real class.

A virtual call: the real object decides which code runs



1. javac sees a.sound() on type Animal and records the method and its slot in the table.
2. At runtime the JVM follows the class pointer in the header of the real object, a Dog.
3. It jumps to the code stored in that slot: Dog.sound, not Animal.sound.

How a virtual method call reaches the right implementation

A call such as `a.sound()` compiles to the bytecode instruction `invokevirtual Animal.sound`. At runtime the JVM reads the object's class pointer, looks up the slot, and runs whichever code that slot holds for **that** class. Interface calls use `invokeinterface` and a slightly slower lookup. HotSpot then speeds the common cases with **inline caches** and **class hierarchy analysis**, so a call site that always sees one class is inlined with a cheap guard.

Bytecode	Used for	Dispatch
`invokevirtual`	Ordinary instance methods	**Dynamic**, by the object's class
`invokeinterface`	Calls through an interface type	Dynamic
`invokespecial`	Constructors, `private` methods, `super.method()`	**Static**: exactly the named method
`invokestatic`	`static` methods	**Static**, by the declared class
`invokedynamic`	Lambdas, string concatenation	Linked at first call by a bootstrap method

What is NOT dispatched dynamically: `static` methods, `private` methods, `final` methods (so devirtualized), constructors, and **fields**.

Fields are not polymorphic

Overriding applies to **methods only**. A field with the same name in the subclass **hides** the parent's field, and which one you get depends on the **reference type**:

```
class P { String name = "P"; String get() { return name; } }
class C extends P { String name = "C"; @Override String get() { return name;
} }

P p = new C();
System.out.println(p.name);        // P    field: chosen by the reference
type P
System.out.println(p.get());       // C    method: chosen by the real object
C
System.out.println(((C) p).name);  // C    now the reference type is C
```

Because of this, avoid same-named fields in a hierarchy; expose data through overridable getters instead.

Method hiding (static) and private methods

```
class A { static String who() { return "A"; } private String secret() {
return "A.secret"; } String call() { return secret(); } }
class B extends A { static String who() { return "B"; } private String
secret() { return "B.secret"; } }

A a = new B();
System.out.println(a.who());     // A        static: bound to the declared
type
System.out.println(a.call());    // A.secret private: not inherited, so no
override; A.call() uses A's own secret()
```

Polymorphism and overloading together

Overload resolution uses **static types**; overriding uses **dynamic types**. Mixing them is a favorite puzzle (see the `A` / `B` example in 3.3.4). A related trap is **double dispatch**: if you need behavior that depends on the runtime types of **two** objects, simple overloading will not do it; use the Visitor pattern or pattern-matching `switch`.

Polymorphism through interfaces and collections

```
List<String> names = new ArrayList<>();      // coded to the interface; the
implementation can change
names = new LinkedList<>();                  // callers using List<String>
are unaffected

List<Payment> queue = List.of(new UpiPayment(), new CardPayment());
queue.forEach(p -> p.pay(50));               // each element behaves
according to its own class
```

The design guideline is "**program to an interface, not an implementation**": declare variables, parameters and return types with the most general suitable type (`List` , `Map` , `Payment`), and name a concrete class only after `new` .

Polymorphism and `instanceof` chains

A chain of `instanceof` checks that picks behavior by type is a **design smell**; put the behavior in the classes and let dispatch choose. For a **closed** set of types (sealed hierarchies, records) a pattern-matching `switch` is the modern alternative and is checked for exhaustiveness (3.5):

```
double area = switch (shape) {
    case Circle c    -> Math.PI * c.r() * c.r();
    case Rectangle r -> r.w() * r.h();
};
```

Benefits, costs and cautions

- **Benefits:** extensibility without modifying existing code, less conditional logic, loose coupling, and easy testing with substitutes (fakes, mocks).
- **Costs:** an extra level of indirection (small, and mostly removed by the JIT) and behavior that is harder to trace by reading code, since the callee depends on the runtime object. A **megamorphic** call site (many different receiver classes) cannot be inlined and is slower.
- **Caution:** never call an overridable method from a constructor (3.1.6), and keep overrides **substitutable** (3.3.6).

Interview traps

- *What is the difference between compile-time and runtime polymorphism?* Overloading vs overriding; compiler vs JVM; declared type vs real type.
- *Are fields polymorphic?* No; field access uses the reference type.
- *Can static methods be polymorphic?* No; they are bound at compile time.
- *Which method runs in* `Animal a = new Dog(); a.eat();`*?* `Dog`'s version if it overrides `eat()`; otherwise the inherited `Animal.eat()`.
- *What happens on a bad downcast?* `ClassCastException`; guard with `instanceof`.
- *How does the JVM find the right method?* Through the object's class pointer and the class's virtual method table (itable for interfaces).

3.3.6 Abstraction in Practice, Composition over Inheritance and Substitutability

Inheritance is powerful and easy to misuse. This section covers how to use it well: abstraction as a design tool, when to choose composition instead, and the Liskov Substitution Principle that tells you whether an inheritance relationship is sound.

Abstraction: expose what, hide how

Abstraction means presenting a simple view of something and hiding the complexity behind it. In Java you build it with **interfaces** and **abstract classes**: callers depend on **what** a type does, never on **how**.

```
// The caller sees only the abstraction
Connection con = dataSource.getConnection();                //
java.sql.Connection is an interface
PreparedStatement ps = con.prepareStatement("select 1");    // PostgreSQL,
MySQL, H2... each supply their own implementation

List<String> names = new ArrayList<>();                     // callers use
List, not ArrayList's array resizing
```

	Abstraction	Encapsulation
Question it answers	*What* can I do with this?	*How is the data protected and kept valid?*
Achieved with	Interfaces, abstract classes	Private fields, access modifiers, validation
Works at	Design of the type system	Design of one class
Benefit	Replaceable implementations, less coupling	Safe state, freedom to change internals

Good abstractions are **small, stable and meaningful**: model the essential operations, name them in the domain's language, and keep implementation details out of the signature.

The Template Method pattern

The parent class fixes the **order of steps** (a `final` method) and leaves selected steps **abstract** for subclasses. It is the standard way to share an algorithm while allowing variation.

```
abstract class DataExporter {
    public final void export(Path target) throws IOException {      // the
fixed skeleton
        List<Row> rows = fetch();
        String content = format(rows);                               //
varies by subclass
        write(target, content);
        afterExport(target);                                         //
optional hook
    }
    protected abstract List<Row> fetch();
    protected abstract String format(List<Row> rows);
    protected void afterExport(Path target) { }                      // hook
with a default (does nothing)
    private void write(Path target, String content) throws IOException {
        Files.writeString(target, content);
    }
}
class CsvExporter extends DataExporter {
    protected List<Row> fetch() { return repo.findAll(); }
    protected String format(List<Row> rows) { return
rows.stream().map(Row::toCsv).collect(Collectors.joining("\n")); }
}
```

Spring's `JdbcTemplate`, `HttpServlet.service()` (calling `doGet`/`doPost`) and JUnit 3's `TestCase` follow this idea. Keep the skeleton `final`, make the varying steps `protected abstract`, and document which hooks are safe to override.

Why "favor composition over inheritance"

Inheritance gives a **compile-time, white-box** relationship: the child sees and depends on the parent's internals, inherits every public method whether wanted or not, and cannot change its parent at runtime. Composition builds behavior by **holding other objects and delegating** to them.

	Inheritance	Composition
Relationship	IS-A, fixed at compile time	HAS-A, can change at runtime
Coupling	Tight: depends on the parent's implementation	Loose: depends on the part's interface
Exposure	Inherits the parent's entire public API	Exposes only what you choose
Reuse of several sources	One parent class only	Any number of collaborators
Testing	Parent logic comes along with the child	Parts are replaceable with fakes
Risk	Fragile base class	Slightly more code to delegate

A classic failure of inheritance (from *Effective Java*): count how many elements were added to a set.

```
class InstrumentedHashSet<E> extends HashSet<E> {
    private int addCount = 0;
    @Override public boolean add(E e) { addCount++; return super.add(e); }
    @Override public boolean addAll(Collection<? extends E> c) { addCount +=
c.size(); return super.addAll(c); }
    public int getAddCount() { return addCount; }
}

InstrumentedHashSet<String> s = new InstrumentedHashSet<>();
s.addAll(List.of("a", "b", "c"));
System.out.println(s.getAddCount());      // 6, not 3!
```

`HashSet.addAll` (inherited from `AbstractCollection`) is implemented by calling `add` for each element, so each element is counted twice: once by the overridden `addAll` and once by the overridden `add`. The subclass depends on an **undocumented implementation detail** of its parent. If a future JDK changed `addAll` to stop calling `add`, the count would silently become correct, or break in a different way.

The composition fix: a wrapper that delegates.

```
class ForwardingSet<E> implements Set<E> {                // delegates every
method to the wrapped set
    private final Set<E> s;
    ForwardingSet(Set<E> s) { this.s = s; }
    public boolean add(E e) { return s.add(e); }
    public boolean addAll(Collection<? extends E> c) { return s.addAll(c); }
    public int size() { return s.size(); }
    // ... every other Set method delegates the same way
}

class InstrumentedSet<E> extends ForwardingSet<E> {
    private int addCount;
    InstrumentedSet(Set<E> s) { super(s); }
    @Override public boolean add(E e) { addCount++; return super.add(e); }
    @Override public boolean addAll(Collection<? extends E> c) { addCount +=
c.size(); return super.addAll(c); }
    int getAddCount() { return addCount; }
}

Set<String> s = new InstrumentedSet<>(new HashSet<>());       // works with
ANY Set, even a TreeSet
s.addAll(List.of("a", "b", "c"));                             // count = 3,
correct
```

Now the count is right because `ForwardingSet.addAll` calls the **wrapped** set's `addAll`, whose internal calls to `add` never reach our override. It also works for **any** `Set`. This is the **Decorator** pattern. Another composition-based pattern is **Strategy**: hold an interface-typed collaborator and swap it at runtime.

```
interface DiscountPolicy { double apply(double total); }
class Cart {
    private DiscountPolicy policy;                            // HAS-A
    Cart(DiscountPolicy policy) { this.policy = policy; }
    void setPolicy(DiscountPolicy p) { this.policy = p; }   // behavior
changes at runtime: impossible with inheritance
    double total(double raw) { return policy.apply(raw); }
}
```

The Liskov Substitution Principle (LSP)

> If `S` is a subtype of `T`, then objects of type `T` can be replaced with objects of type `S` **without breaking the correctness of the program**.

IS-A in Java syntax is not enough. The subclass must also honor the **behavioral contract** of the parent:

- It must **not strengthen preconditions** (demand more from callers than the parent did).
- It must **not weaken postconditions** (promise less than the parent did).
- It must **preserve the parent's invariants**.
- It must **not throw new kinds of exceptions** the parent's contract did not allow.

The classic violation: `Square extends Rectangle`.

```
class Rectangle {
    protected int w, h;
    void setWidth(int w)  { this.w = w; }
    void setHeight(int h) { this.h = h; }
    int area() { return w * h; }
}
class Square extends Rectangle {                          // mathematically a
square IS-A rectangle
    @Override void setWidth(int w)  { this.w = w; this.h = w; }    // keeps
it square
    @Override void setHeight(int h) { this.w = h; this.h = h; }
}

static void stretch(Rectangle r) {
    r.setWidth(5);
    r.setHeight(4);
    assert r.area() == 20;          // true for a Rectangle; fails for a
Square (area 16)
}
```

The code written for `Rectangle` breaks when given a `Square`, so `Square` is **not** a valid substitute. The mutable setters make the contract "width and height vary independently". Fix by removing the false hierarchy: make both implement a `Shape` with an `area()`, make them immutable, or use composition.

More signs of an LSP violation

- An override that **throws** `UnsupportedOperationException` or does nothing (a `ReadOnlyList.add` that throws breaks code that expects a `List` to accept adds; this is a known tension in the JDK's own immutable collections, handled by documenting `add` as optional).
- Code that needs `instanceof` checks on a parent-typed reference to avoid certain children.
- A child that restricts accepted inputs more than the parent did.

Covariance rules that Java enforces for you: return types may narrow, parameter types may not (they would be overloads), checked exceptions may narrow. These compile-time rules are a partial enforcement of LSP; behavior is up to you and your tests.

The fragile base class problem

The `InstrumentedHashSet` bug shows it: the parent's **implementation** changes (or was never documented) and children break even though the parent's API did not change. Causes and defenses:

Cause	Defense
A subclass depends on **which parent methods call which** (self-use)	Document self-use of overridable methods, or avoid overridable calls inside the parent
Parent adds a new method that clashes with a child's	Keep hierarchies you do not own shallow; prefer composition for third-party types
Parent constructors call overridable methods (3.1.6)	Never do that
Wide `protected` surfaces expose internals	Keep `protected` members to a minimum; they are a public contract too
Subclass and parent evolve in different release cycles	Use `final` or `sealed`, or ship the parent only as an interface

Designing for inheritance: a checklist

1. **Prohibit it by default.** Make classes `final` or `sealed` unless you deliberately design them for extension.

2. If you allow it, **document exactly which methods are safe to override**, including which other methods they call.
3. Provide few, well-defined **`protected` hooks** instead of exposing fields.
4. **Never call overridable methods** from constructors, `clone()` or deserialization code.
5. Prefer an **interface plus an optional skeletal abstract class** (3.2.3, 3.4) over a concrete base class.
6. Test every subclass against the **parent's contract**, not just its own behavior.

When inheritance is the right tool

Use it when there is a genuine **IS-A** relationship, the subclass honors LSP, parent and child are maintained **together** (same module or team), and you gain real polymorphism. Typical good uses: abstract skeletons such as `AbstractList`, framework extension points designed for subclassing (`HttpServlet`, `OncePerRequestFilter`), exception hierarchies, and closed families modeled with `sealed` types.

Interview traps

- *Why prefer composition over inheritance?* Looser coupling, hidden internals, runtime flexibility, no fragile-base-class risk, and the ability to combine several collaborators.
- *Is `Stack extends Vector` a good design?* No: a stack is not a vector, and callers can bypass stack behavior through the inherited API. The JDK recommends `Deque`.
- *State the Liskov Substitution Principle.* Subtypes must be usable wherever their parent type is expected, without altering the program's correctness.
- *Why does `Square extends Rectangle` violate LSP?* Because of the behavioral contract that width and height are independent.
- *What is the Template Method pattern?* A parent `final` method defines the algorithm's steps and calls abstract or hook methods that subclasses implement.
- *How does composition avoid the instrumented-set bug?* The wrapper delegates to the contained object, so the parent's internal self-calls never reach the wrapper's overrides.

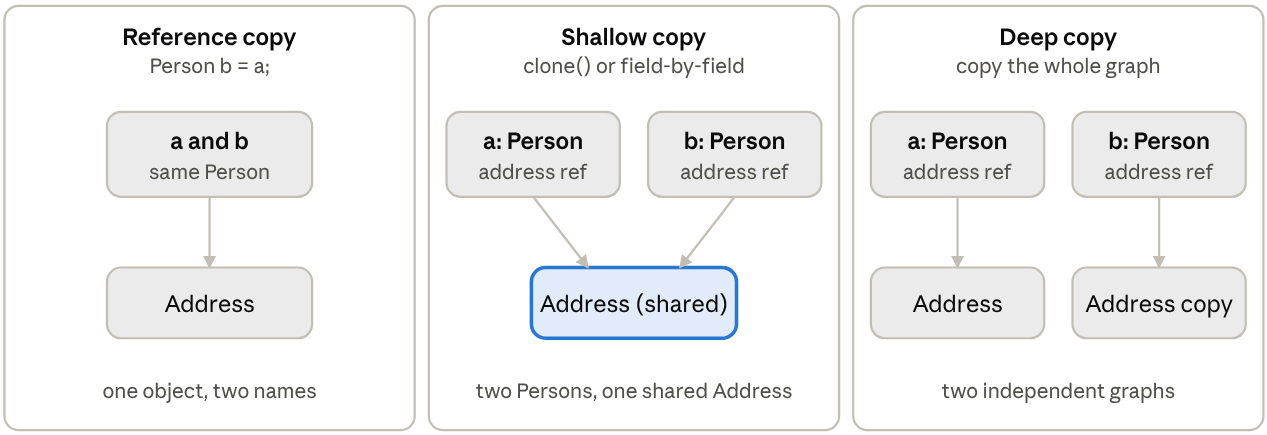
3.3.7 Shallow Copy vs Deep Copy

A copy of an object can be made at three depths, and they behave very differently once someone changes something. This section builds directly on references and pass-by-value (3.1.6).

Assignment copies the reference, not the object

```
Person a = new Person("Asha", new Address("Pune"));
Person b = a;                  // reference copy: b and a point to the SAME
object (aliasing)
b.name = "Ravi";
System.out.println(a.name);    // Ravi: there is only one Person
```

What a copy shares: one object, one inner object, or nothing



Reference copy, shallow copy and deep copy

The drawing shows the three depths. A **shallow** copy creates a new outer object but reuses the objects it refers to. A **deep** copy duplicates the whole **graph** of objects reachable through mutable references.

	Reference copy	Shallow copy	Deep copy
New outer object?	No	**Yes**	**Yes**
Primitive fields	Shared (same object)	Copied	Copied
Referenced objects	Shared	**Shared**	**Duplicated**
Change a nested object through the copy	Affects the original	**Affects the original**	Does not affect the original
Cost	Free	Cheap	Proportional to graph size

The classes used below

```
class Address {
    String city;
    Address(String city) { this.city = city; }
}
class Person {
    String name;                // String is immutable: sharing it is safe
    int age;                    // primitive: copied by value
    Address address;            // mutable object: sharing it matters
    List<String> tags = new ArrayList<>();
    Person(String name, int age, Address address) { this.name = name;
this.age = age; this.address = address; }
}
```

Shallow copy

A shallow copy copies each **field value**: primitives are duplicated, **references are copied as references**, so both objects now point to the same nested objects.

```
Person a = new Person("Asha", 30, new Address("Pune"));

Person shallow = new Person(a.name, a.age, a.address);   // new Person, same
Address object

shallow.name = "Ravi";            // String is replaced by a new String in
'shallow' only
shallow.age = 31;                 // primitive: independent
shallow.address.city = "Delhi";   // changes the Address that BOTH objects
share

System.out.println(a.name);           // Asha   (unaffected)
System.out.println(a.age);            // 30     (unaffected)
System.out.println(a.address.city);   // Delhi  (the shared Address changed!)
```

Is a shallow copy always a bug? No. If the shared parts are **immutable** (`String` , wrappers, `LocalDate` , records of immutable parts), sharing them is safe and efficient. The danger is **shared mutable state**.

Shallow copies appear everywhere:

```
List<Person> copy = new ArrayList<>(original);     // new list, SAME Person
objects
List<Person> copy2 = List.copyOf(original);        // immutable snapshot,
SAME Person objects
Person[] arr2 = Arrays.copyOf(arr, arr.length);    // new array, SAME
elements
Person[] arr3 = arr.clone();                       // same
```

Changing a `Person` through `copy.get(0)` changes it in `original` too.

Deep copy

A deep copy duplicates every **mutable** object reachable from the original, recursively, so the two graphs are completely independent. There are four common techniques.

1. Copy constructor or copy factory (recommended). You write explicit code that copies each field correctly.

```
class Address {
    String city;
    Address(String city) { this.city = city; }
    Address(Address other) { this.city = other.city; }                 //
copy constructor
}
class Person {
    String name; int age; Address address; List<String> tags = new
ArrayList<>();
    Person(String name, int age, Address address) { this.name = name;
this.age = age; this.address = address; }

    Person(Person other) {                                             //
deep copy constructor
        this.name = other.name;                                        //
immutable: share
        this.age = other.age;                                          //
primitive: copy
        this.address = other.address == null ? null : new
Address(other.address);   // mutable: duplicate
        this.tags = new ArrayList<>(other.tags);                       //
collection of immutable Strings: new list is enough
    }
}
```

```
Person a = new Person("Asha", 30, new Address("Pune"));
Person deep = new Person(a);
deep.address.city = "Delhi";
System.out.println(a.address.city);   // Pune: independent
```

Advantages: it uses ordinary constructors (no hidden magic), works with `final` fields, handles `null` explicitly, and each class owns the rule for its own copy. It is the technique recommended in *Effective Java*.

2. `clone()` and `Cloneable`.

```
class Person implements Cloneable {
    String name; Address address; int[] scores;

    @Override public Person clone() {                          // widen to
public, narrow the return type
        try {
            Person p = (Person) super.clone();                 // shallow
copy of all fields, no constructor call
            p.address = new Address(address);                  // then fix up
each mutable field: deep
            p.scores  = scores.clone();                        // arrays:
clone() is the idiom
            return p;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);                       // cannot
happen: we implement Cloneable
        }
    }
}
```

How it works: `Object.clone()` is `protected` and `native`. It allocates a new object of the **same runtime class** and copies every field **bit for bit** (a shallow copy) **without calling any constructor**. It throws `CloneNotSupportedException` unless the class implements the **marker interface** `Cloneable`. You then replace mutable fields with copies.

`clone()` is widely considered **flawed**: `Cloneable` contains no `clone()` method, so the interface does not describe the contract; constructors are bypassed, so invariants set there are skipped; `final` fields cannot be reassigned in a fixed-up clone; subclasses must keep `super.clone()` chains correct; and a forgotten fix-up silently shares mutable state. The one safe and idiomatic use is **array cloning** (`int[] b = a.clone();` , a shallow copy). Prefer copy constructors for your own classes.

3. Serialization round trip. Write the object graph to bytes and read it back; the whole graph, including cycles and shared sub-objects, is rebuilt.

```
@SuppressWarnings("unchecked")
static <T extends Serializable> T deepCopy(T obj) throws IOException,
ClassNotFoundException {
    ByteArrayOutputStream bos = new ByteArrayOutputStream();
    try (ObjectOutputStream out = new ObjectOutputStream(bos)) {
out.writeObject(obj); }
    try (ObjectInputStream in = new ObjectInputStream(new
ByteArrayInputStream(bos.toByteArray()))) {
        return (T) in.readObject();
    }
}
```

The whole graph must be `Serializable`, `transient` fields are lost, it is **slow**, and Java deserialization is a well-known security risk when the bytes come from outside (here they are produced and consumed in-process, which is safe). It is a convenience for tests and quick tools, not a production default.

4. A mapping library or JSON round trip (Jackson `readValue(writeValueAsString(obj), Person.class)`, MapStruct, ModelMapper). Convenient, but costly and sensitive to configuration.

And a **fifth path that avoids copying altogether: immutability.** If `Person` and `Address` are immutable, nobody can change a shared object, so sharing is always safe and a "copy" is just a reference (Part 3.5).

Deep copy pitfalls

- **Cycles:** if `A` refers to `B` and `B` back to `A`, naive recursive copying never ends. Copy code must keep an identity map of already-copied objects (serialization does this automatically).
- **Shared sub-objects:** if two fields point to the **same** `Address`, a naive deep copy makes two separate copies, changing the shape of the graph.
- **Cost:** a deep copy of a large graph is O(size of graph) in time and memory.
- **Non-copyable state:** threads, open files, sockets and locks cannot be meaningfully duplicated.
- **How deep is deep enough?** Copy only what is **mutable**. `String`, boxed numbers, `LocalDate`, enums and records of immutable fields need no copy.

Defensive copying

The same techniques protect class invariants (3.1.3): copy mutable arguments **on the way in** and mutable internals **on the way out**.

```
public final class Booking {
    private final Date start;                                    // Date
is mutable
    public Booking(Date start) { this.start = new Date(start.getTime()); }
// copy in
    public Date getStart() { return new Date(start.getTime()); }
// copy out
}
```

Choosing a technique

Situation	Use
Share data and nobody mutates it	A reference copy; make the type immutable
Copy a collection of immutable elements	`new ArrayList<>(list)` or `List.copyOf`
Fully independent copy of your own class	**Copy constructor or copy factory**
Copy an array	`arr.clone()` or `Arrays.copyOf`
One-off deep copy of a big graph in a test	Serialization or a JSON round trip
Protect an invariant	Defensive copies at the boundary

Interview traps

- *Does `clone()` make a deep copy?* No, a shallow copy unless you override it to fix up mutable fields.
- *Does `new ArrayList<>(list)` copy the elements?* It copies the list; the **elements** are shared (shallow).
- *Does `clone()` call the constructor?* No.
- *Why is `Cloneable` considered broken?* It is an empty marker, bypasses constructors, conflicts with `final` fields and invites shared-state bugs.

- *Is copying an `int[]` with `clone()` deep?* For primitives, yes (the values are copied). For an `Object[]` or `int[][]` it copies only the references, so the inner objects or arrays are shared.
- *How do you deep-copy a `List<Person>`?* Loop or stream and copy each element: `list.stream().map(Person::new).collect(Collectors.toList())` with a copy constructor.

Review Questions: Part 3.3

Attempt every question first. Part A is multiple choice (many are predict-the-output), Part B is short answer, Part C is coding and prediction, and Part D is senior level. All answers and explanations are consolidated in the next section.

Part A: Multiple Choice

Q1. A subclass object is created. Which statement about the parent's `private` fields is true?

- (a) They are not part of the object
- (b) They exist in the object but cannot be accessed by name from the subclass
- (c) They are inherited and directly accessible
- (d) They are copied into the subclass as `protected`

Q2. What happens with `class C extends A, B { }` where `A` and `B` are classes?

- (a) Compiles
- (b) Compile error
- (c) Compiles with a warning
- (d) Runtime error

Q3. Which of these is NOT inherited by a subclass?

- (a) A `public` method
- (b) A `protected` field
- (c) A constructor
- (d) A `default` method of an implemented interface

Q4. A `Car` creates its own `Engine` in its constructor, never exposes it, and the engine has no meaning outside the car. This relationship is:

- (a) Association
- (b) Aggregation
- (c) Composition
- (d) Inheritance

Q5. A `Team` receives a list of `Player` objects from outside, and the players keep existing if the team is discarded. This relationship is:

- (a) Aggregation
- (b) Composition
- (c) Inheritance
- (d) Overriding

Q6. What is printed?

```
class P { String n = "P"; String g() { return n; } }
class C extends P { String n = "C"; @Override String g() { return n; } }
P p = new C();
System.out.println(p.n + p.g());
```

- (a) `PP`
- (b) `CC`
- (c) `PC`
- (d) `CP`

Q7. What is printed?

```
class A { static String who() { return "A"; } }
class B extends A { static String who() { return "B"; } }
A a = new B();
System.out.println(a.who());
```

- (a) `A`
- (b) `B`
- (c) Compile error
- (d) `null`

Q8. What is printed?

```
static void f(long x)    { System.out.print("long "); }
static void f(Integer x) { System.out.print("Integer "); }
static void f(int... x)  { System.out.print("varargs "); }
f(5);
```

- (a) `long`
- (b) `Integer`
- (c) `varargs`
- (d) Compile error

Q9. What is printed?

```
static void g(Object o) { System.out.print("Object "); }
static void g(String s) { System.out.print("String "); }
g(null);
```

- (a) `Object`
- (b) `String`
- (c) Compile error (ambiguous)
- (d) `NullPointerException`

Q10. What happens?

```
static void h(Integer i) { }
static void h(String s)  { }
h(null);
```

- (a) Calls `h(Integer)`
- (b) Calls `h(String)`
- (c) Compile error (ambiguous)
- (d) Runtime error

Q11. The parent declares `public void run()`. A child overrides it as `protected void run()`. What happens?

- (a) Compiles
- (b) Compile error: access cannot be reduced
- (c) The child method is ignored
- (d) Runtime error

Q12. Which override is legal if the parent declares `Animal reproduce()`?

- (a) `Dog reproduce()` (where `Dog extends Animal`)
- (b) `Object reproduce()`
- (c) `void reproduce()`
- (d) `int reproduce()`

Q13. A parent method declares `throws IOException`. Which override is legal?

- (a) `throws Exception`
- (b) `throws FileNotFoundException`
- (c) `throws Throwable`
- (d) None of these

Q14. What happens?

```
class Point {
    int x, y;
    @Override public boolean equals(Point p) { return x == p.x && y == p.y; }
}
```

- (a) Compiles and overrides `equals`
- (b) Compile error: method does not override a superclass method
- (c) Compiles, but is an overload
- (d) Runtime error

Q15. What is printed?

```
class A { void f(Object o) { System.out.println("A.f(Object)"); } }
class B extends A {
    @Override void f(Object o) { System.out.println("B.f(Object)"); }
    void f(String s)           { System.out.println("B.f(String)"); }
}
A a = new B();
a.f("x");
```

- (a) `A.f(Object)`
- (b) `B.f(Object)`
- (c) `B.f(String)`
- (d) Compile error

Q16. What is printed?

```
static void show(Object o) { System.out.print("Object "); }
static void show(String s) { System.out.print("String "); }
Object x = "hello";
show(x);
```

- (a) `Object`
- (b) `String`
- (c) Compile error
- (d) `ClassCastException`

Q17. Runtime polymorphism is achieved through:

- (a) Method overloading
- (b) Method overriding
- (c) Constructor overloading
- (d) Operator overloading

Q18. A `static` method call is bound:

- (a) At runtime, by the object's class
- (b) At compile time, by the declared type
- (c) By the garbage collector
- (d) At class-loading time, by the ClassLoader

Q19. What happens?

```
interface A { default String hello() { return "A"; } }
interface B { default String hello() { return "B"; } }
class C implements A, B { }
```

- (a) Compiles; `hello()` returns `"A"`
- (b) Compile error: `C` must override `hello()`
- (c) Compiles; `hello()` returns `"B"`
- (d) Runtime error when `hello()` is called

Q20. Which call is NOT resolved by dynamic dispatch?

- (a) A call to an overridable instance method
- (b) A call through an interface reference

- (c) A call to a `static` method
- (d) A call to an abstract method

Q21. What happens?

```
Animal a = new Cat();   // Cat and Dog both extend Animal
Dog d = (Dog) a;
```

- (a) Compile error
- (b) `ClassCastException` at runtime
- (c) `d` is `null`
- (d) Compiles and runs without error

Q22. What is `original.get(0).name` afterwards?

```
List<Person> original = new ArrayList<>(List.of(new Person("Asha")));
List<Person> copy = new ArrayList<>(original);
copy.get(0).name = "Ravi";
```

- (a) `Asha`
- (b) `Ravi`
- (c) `null`
- (d) Compile error

Q23. `Object.clone()` produces which kind of copy by default?

- (a) Deep copy
- (b) Shallow copy
- (c) Reference copy only
- (d) No copy; it returns the same object

Q24. Does `Object.clone()` call the class's constructor?

- (a) Yes, the no-arg constructor
- (b) Yes, the most specific constructor
- (c) No
- (d) Only for final classes

Q25. `int[][] b = a.clone();` where `a` is `int[][]`. What is shared?

- (a) Nothing

- (b) The inner `int[]` row arrays
- (c) Only the first row
- (d) Every `int` value is shared

Q26. Which design most clearly violates the Liskov Substitution Principle?

- (a) `ArrayList` extends `AbstractList`
- (b) `Square extends Rectangle` with independent `setWidth` and `setHeight` that keep the shape square
- (c) `Dog extends Animal` and overrides `sound()`
- (d) `FileLoader` overrides `load()` to throw a narrower exception

Part B: Short Answer

Q27. Distinguish IS-A, HAS-A and "uses" relationships, and give a Java form and an example of each.

Q28. Explain the difference between aggregation and composition in terms of lifetime, ownership and creation. How would you recognize each in code?

Q29. Why does Java not allow multiple inheritance of classes? How do interfaces and default methods provide a safe substitute, and what are the conflict rules?

Q30. Name the five types of inheritance and state which Java supports with classes and which with interfaces.

Q31. Compare overloading and overriding, including binding time, the signature rules, return types, access and exceptions.

Q32. Explain the three phases the compiler uses to resolve an overloaded call, and say why `f(5)` picks `f(long)` over `f(Integer)`.

Q33. What does `@Override` do? Describe a real bug it prevents.

Q34. Explain how dynamic method dispatch works inside the JVM.

Q35. Why are fields not polymorphic? Show an example and describe the design consequence.

Q36. Explain shallow copy and deep copy, list four ways to make a deep copy, and say why `clone()` is considered flawed.

Q37. Give four reasons to prefer composition over inheritance, and state when inheritance is still right.

Q38. State the Liskov Substitution Principle and its behavioral rules. Why does `Square extends Rectangle` break it?

Part C: Coding and Prediction

C1. Design an abstract class `Vehicle` with a `String id`, an abstract method `double fuelCost(double km)` and a concrete `describe()`. Add `Car` (12 per km) and `Truck` (30 per km plus a 100 handling fee). Process a `List<Vehicle>` polymorphically and print the total for 50 km.

C2. Predict the exact output:

```
class A {
    void m(int x)  { System.out.print("A.int "); }
    void m(long x) { System.out.print("A.long "); }
}
class B extends A {
    @Override void m(int x)  { System.out.print("B.int "); }
    void m(double x)         { System.out.print("B.double "); }
}
A a = new B();
B b = new B();
a.m(1);
a.m(1L);
b.m(1);
b.m(1L);
b.m(1.5f);
```

C3. This class misuses inheritance. Rewrite it with composition so that it exposes only `push`, `pop`, `peek`, `isEmpty` and `size`:

```
class CountingStack extends ArrayList<String> {
    void push(String s) { add(s); }
    String pop() { return remove(size() - 1); }
}
```

C4. Implement the Template Method pattern: an abstract `ReportGenerator` with a `final String generate()` that returns header, body and footer; abstract `title()` and `body()`; an overridable `footer()` hook with a default. Add `SalesReport` and `InventoryReport`.

C5. Write an `Order` class holding a `Customer` (with a mutable `String city`) and a `List<OrderLine>` (each with `sku` and `qty`). Give each class a copy constructor so that `new Order(original)` is a fully independent deep copy, then show that changing the copy does not change the original.

C6. Predict the exact output:

```
class Base {
    Base() { init(); }
    void init() { System.out.println("Base.init"); }
}
class Derived extends Base {
    String s = "ready";
    Derived() { super(); System.out.println("s=" + s); }
    @Override void init() { System.out.println("Derived.init s=" + s); }
}
new Derived();
```

C7. Make this compile and have `C.hello()` return `"A+B"` by calling both interface versions:

```
interface A { default String hello() { return "A"; } }
interface B { default String hello() { return "B"; } }
class C implements A, B { }
```

Part D: Senior Level

S1. For `A a = new B(); a.f("x");` (the overload/override puzzle), explain exactly what the compiler records in the bytecode and what the JVM does at runtime.

S2. Compare `invokestatic`, `invokespecial`, `invokevirtual` and `invokeinterface`. How does HotSpot optimize virtual calls (class hierarchy analysis, inline caches), and what is a megamorphic call site?

S3. Explain "design and document for inheritance, or else prohibit it". What must the documentation of an overridable method say?

S4. Go deeper into the Liskov Substitution Principle: the rules for preconditions, postconditions, invariants and exceptions, how Java's type system enforces part of them, and how you would test that a subclass honors its parent's contract.

S5. You must add logging and metrics to a `PaymentRepository`. Compare subclassing it, writing a Decorator, and using Spring AOP. Which would you choose and why?

S6. Spring creates CGLIB (subclass) proxies for `@Transactional` beans without an interface. Explain how this works, and why `final` classes, `final` methods and `private` methods cause transactions to silently not apply.

S7. Explain the diamond problem for default methods, the three resolution rules, and what happens when an abstract class and an interface both provide a method with the same signature.

S8. What are bridge methods? Show what `javac` generates for a covariant return and for `class Name implements Comparable<Name>`.

S9. Design a generic deep-copy for an object graph that may contain cycles and shared sub-objects. What data structure makes it correct, and when would you avoid deep copying altogether?

S10. A parent class implements `clone()` by calling `new Parent(...)` instead of `super.clone()`. Show how a subclass's `clone()` then breaks, and list the Bloch guidelines for `clone`.

S11. Explain the performance side of polymorphism, and compare a sealed hierarchy with a pattern-matching `switch` against virtual dispatch for adding new operations versus adding new types.

S12. Why are `protected` fields and field hiding considered harmful? Redesign a hierarchy that exposes `protected double balance` into a safer one.

Answers: Part 3.3

Part A: Multiple Choice

Q	Answer	Explanation
1	(b)	Private fields are stored in every subclass object but are not accessible by name from the subclass.
2	(b)	A class can extend only one class.
3	(c)	Constructors are never inherited; each class defines its own.
4	(c)	The whole creates and owns the part, which has no life outside it: composition.

Q	Answer	Explanation
5	(a)	Parts come from outside and outlive the whole: aggregation.
6	(c) `PC`	`p.n` is a field, chosen by the reference type `P`; `p.g()` is dispatched to `C.g()`, which returns `C`'s own `n`.
7	(a) `A`	Static methods are bound by the declared type.
8	(a)	Phase 1 allows primitive widening (`int` to `long`) but not boxing, so `f(long)` wins before `f(Integer)` is considered.
9	(b)	Both apply to `null`; `String` is more specific than `Object`.
10	(c)	`Integer` and `String` are unrelated, so neither is more specific.
11	(b)	An override may not reduce visibility.
12	(a)	Covariant returns allow a subtype. Primitives and `void` must match exactly, and `Object` is a supertype.
13	(b)	An override may throw the same checked exception or a narrower one, never a broader one.
14	(b)	The parameter is `Point`, not `Object`, so it does not override `equals(Object)`, and `@Override` turns that into a compile error.
15	(b)	The compiler chooses the signature `f(Object)` from type `A`; at runtime `B`'s override of that signature runs. `f(String)` is invisible through an `A` reference.
16	(a)	The overload is chosen from the **compile-time** type `Object`, not the runtime `String`.
17	(b)	Overriding is dispatched at runtime. Overloading is compile-time.

Q	Answer	Explanation
18	(b)	`static` calls are resolved at compile time from the declared class.
19	(b)	Two unrelated defaults conflict, so `C` must override and choose.
20	(c)	Static calls (`invokestatic`) are not dynamically dispatched.
21	(b)	The cast compiles because `Dog` is an `Animal`, but the real object is a `Cat`, so it throws at runtime.
22	(b) `Ravi`	The new list is a separate list, but both lists hold references to the same `Person` objects.
23	(b)	`Object.clone()` copies fields bit for bit: a shallow copy.
24	(c)	`clone()` allocates and copies without invoking any constructor.
25	(b)	The outer array is copied; the inner `int[]` row arrays are shared.
26	(b)	Code written for `Rectangle` assumes width and height are independent; `Square` breaks that contract.

Part B: Short Answer (model answers)

Q27. IS-A means a subtype relationship, expressed with `extends` or `implements` (`Dog` IS-A `Animal`). **HAS-A** means one object holds another as a part, expressed with a field (`Car` HAS-A `Engine`); it comes as aggregation or composition. **Uses** is a temporary association or dependency, expressed with a parameter, local variable or return type (`Doctor.treat(Patient p)`).

Q28. In **aggregation** the whole does not own the parts: they are created outside and passed in, can be shared, and outlive the whole (`Team` with `Player`s). In **composition** the whole owns the parts: it creates them, does not share them, and they are destroyed with

it (`House` with `Room`s). In code, aggregation shows as a constructor or setter that receives existing objects; composition shows as the whole calling `new` for the part itself and not exposing it. The test: "if the whole is deleted, should the part go too?"

Q29. With two parent classes, a method or field inherited from a common ancestor can arrive by two paths (the diamond problem): which implementation runs, and is there one copy of the state or two? Java avoids this by allowing a class one parent class only. A class can still implement many **interfaces**, which carry contract and (since Java 8) `default` methods but **no instance state**. If two interfaces supply conflicting defaults, the rules are: (1) a class method (even inherited) wins; (2) otherwise the more specific interface wins; (3) otherwise the compiler forces you to override, optionally calling `X.super.m()`.

Q30. Single (one parent), multilevel (a chain), hierarchical (one parent, many children): all allowed with classes and interfaces. Multiple (one child, many parents) and hybrid (a combination that includes it): **not** allowed with classes, allowed with interfaces.

Q31. *Overloading:* same name, different parameter list, in one class or across a hierarchy; chosen by the compiler from the declared argument types (compile-time polymorphism); return type, access and exceptions may differ freely. *Overriding:* a subclass redefines an inherited instance method with the same signature; chosen at runtime from the object's class; the return type must be the same or covariant, access the same or wider, checked exceptions the same or narrower; `static`, `private` and `final` methods cannot be overridden.

Q32. Phase 1 allows only subtyping and primitive widening. Phase 2 adds boxing and unboxing. Phase 3 adds varargs. The compiler stops at the first phase with a match and then picks the **most specific** method; if none is most specific it is an ambiguity error. `f(5)` matches `f(long)` in phase 1 (widening), so `f(Integer)`, which needs boxing (phase 2), is never reached.

Q33. `@Override` makes the compiler check that the method really overrides or implements a supertype method. Without it, a wrong parameter type silently creates an **overload**. The classic bug is `equals(Point p)` instead of `equals(Object o)`: the code compiles, but `HashSet` and `List.contains` still use `Object`'s identity-based `equals`, so lookups fail.

Q34. Each class has a **virtual method table**: a slot per overridable instance method. A subclass copies its parent's table and replaces the slots it overrides. Each object's header holds a class pointer. `invokevirtual` finds the object's class, indexes the table slot, and

calls the code in that slot. Interface calls use a similar lookup (`invokeinterface`). HotSpot optimizes this with class hierarchy analysis and inline caches, so a site that sees one class is inlined with a cheap guard.

Q35. Only methods are dispatched by the object's class. A field access is resolved at compile time from the reference's declared type, so `P p = new C(); p.n` reads `P.n` even though the object is a `C` that also declares `n` . The consequence: avoid same-named fields in a hierarchy, keep fields `private` , and expose data through overridable methods.

Q36. A **shallow copy** creates a new outer object but shares everything it references; a **deep copy** also duplicates the referenced mutable objects, recursively. Deep copy techniques: (1) copy constructor or copy factory; (2) a `clone()` that fixes up each mutable field; (3) a serialization round trip; (4) a mapping or JSON library. `clone()` is flawed because `Cloneable` is an empty marker with no `clone` method, `Object.clone()` is protected, it bypasses constructors, it clashes with `final` fields, it shares mutable state by default, and subclasses must cooperate. Prefer copy constructors; use `clone()` only to copy arrays.

Q37. (1) **Looser coupling:** you depend on the part's interface, not its implementation. (2) **Encapsulation:** you expose only the methods you choose, not an inherited API. (3) **Runtime flexibility:** a part can be replaced while the program runs (Strategy). (4) **No fragile base class:** parent changes do not silently break you; and you can combine many collaborators, while a class has one parent. Inheritance is still right for a true IS-A with substitutability, in a hierarchy maintained together, such as abstract skeletons, framework extension points and exception or sealed families.

Q38. If `S` is a subtype of `T` , a `T` can be replaced by an `S` without breaking the program. Behaviorally: do not strengthen preconditions, do not weaken postconditions, keep the invariants, and do not throw new exception types. `Square extends Rectangle` breaks it because `Rectangle` 's contract is that `setWidth` leaves the height alone; a `Square` must change both, so code that sets width 5 and height 4 expecting area 20 gets 16.

Part C: Coding and Prediction (model solutions)

C1. Vehicles

```
abstract class Vehicle {
    private final String id;
    protected Vehicle(String id) { this.id = id; }
    abstract double fuelCost(double km);
    String describe(double km) { return id + " costs " + fuelCost(km) + " for
" + km + " km"; }
}
class Car extends Vehicle {
    Car(String id) { super(id); }
    @Override double fuelCost(double km) { return 12 * km; }
}
class Truck extends Vehicle {
    Truck(String id) { super(id); }
    @Override double fuelCost(double km) { return 30 * km + 100; }
}

List<Vehicle> fleet = List.of(new Car("C1"), new Truck("T1"));
fleet.forEach(v -> System.out.println(v.describe(50)));    // C1 costs 600.0
...  T1 costs 1600.0 ...
double total = fleet.stream().mapToDouble(v -> v.fuelCost(50)).sum();   //
2200.0
```

C2. Output: `B.int A.long B.int A.long B.double`

- `a.m(1)`: the static type is `A`; both `m(int)` and `m(long)` apply and `m(int)` is more specific. The runtime object is a `B`, which overrides `m(int)`: `B.int`.
- `a.m(1L)`: only `m(long)` applies; `B` does not override it: `A.long`.
- `b.m(1)`: `B` has `m(int)` (override), `m(long)` and `m(double)`; `m(int)` is most specific: `B.int`.
- `b.m(1L)`: `m(long)` and `m(double)` both apply by widening; `long` is more specific than `double`: `A.long`.
- `b.m(1.5f)`: `float` widens only to `double`, so `m(double)`: `B.double`.

C3. Composition

```
class CountingStack {
    private final Deque<String> items = new ArrayDeque<>();
    void push(String s) { items.push(s); }
    String pop()        { return items.pop(); }
    String peek()       { return items.peek(); }
    boolean isEmpty()   { return items.isEmpty(); }
    int size()          { return items.size(); }
}
```

Callers can no longer call `add(0, x)`, `remove(3)` or `clear()` through the stack; only stack operations exist.

C4. Template Method

```
abstract class ReportGenerator {
    public final String generate() {                // fixed skeleton
        return title() + "\n" + body() + "\n" + footer();
    }
    protected abstract String title();
    protected abstract String body();
    protected String footer() { return "-- end --"; }   // hook with a
default
}
class SalesReport extends ReportGenerator {
    protected String title() { return "Sales"; }
    protected String body()  { return "Total: 1200"; }
}
class InventoryReport extends ReportGenerator {
    protected String title() { return "Inventory"; }
    protected String body()  { return "Items: 340"; }
    @Override protected String footer() { return "-- counted at 18:00 --"; }
}
```

C5. Deep-copying an order

```
class Customer {
    String city;
    Customer(String city) { this.city = city; }
    Customer(Customer o)  { this.city = o.city; }
}
class OrderLine {
    final String sku; int qty;
    OrderLine(String sku, int qty) { this.sku = sku; this.qty = qty; }
```

```
    OrderLine(OrderLine o)         { this(o.sku, o.qty); }
}
class Order {
    Customer customer;
    List<OrderLine> lines = new ArrayList<>();
    Order(Customer c) { this.customer = c; }
    Order(Order o) {                                          // deep copy
constructor
        this.customer = new Customer(o.customer);
        for (OrderLine l : o.lines) this.lines.add(new OrderLine(l));
    }
}

Order original = new Order(new Customer("Pune"));
original.lines.add(new OrderLine("A1", 2));
Order copy = new Order(original);
copy.customer.city = "Delhi";
copy.lines.get(0).qty = 9;
System.out.println(original.customer.city);       // Pune
System.out.println(original.lines.get(0).qty);    // 2
```

C6. Output

```
Derived.init s=null
s=ready
```

`Base()` runs first and calls `init()`, which dispatches to `Derived`'s override. `Derived`'s field initializer (`s = "ready"`) has not run yet, so `s` is `null`. After `super()` returns, the initializer runs and the constructor body prints `s=ready`.

C7. Resolving the conflict

```
class C implements A, B {
    @Override public String hello() { return A.super.hello() + "+" +
B.super.hello(); }   // "A+B"
}
```

Part D: Senior Level (model answers)

S1. At **compile time**, the variable's type is `A`, so the compiler looks up `f` in `A`, finds only `f(Object)`, and emits `invokevirtual A.f:(Ljava/lang/Object;)V`. The **descriptor** (name plus erased parameter types) is fixed here. At **runtime** the JVM resolves that

method reference, finds the receiver's class `B`, and selects the method in `B` (or its parents) with the **same name and descriptor**: `B.f(Object)` overrides it, so that runs. `B.f(String)` has a different descriptor (`(Ljava/lang/String;)V`) and is never considered. With `B b = new B(); b.f("x")` the static type `B` offers `f(String)` as the most specific match, so the emitted descriptor is the `String` one.

S2. `invokestatic` calls a static method and is bound by class. `invokespecial` calls constructors and `super.method()` exactly as named, without virtual lookup (before Java 11 it was also used for `private` methods; since nestmates, `javac` emits `invokevirtual` for private methods, which the JVM still binds non-virtually). `invokevirtual` selects through the class's vtable. `invokeinterface` selects through the interface table, a bit slower. HotSpot speeds virtual calls with **class hierarchy analysis** (if a method has only one implementation among loaded classes, inline it, with a dependency that triggers deoptimization when a new subclass loads) and **inline caches** (profile the receiver types at the call site: **monomorphic**, one type, inlined behind a type check; **bimorphic**, two types). A **megamorphic** site (three or more receiver classes) falls back to a real table call and **cannot be inlined**, which also blocks follow-on optimizations such as escape analysis. Mitigations: keep hot loops over one type, split work by type, or restructure with sealed types and a `switch`.

S3. Subclasses depend on a parent's behavior, so the parent's documentation is part of its contract. For each overridable method, document **what it does**, and its **self-use**: which other overridable methods it calls and in which order (Java's `@implSpec` Javadoc tag exists for this), so subclass authors know what overriding will affect. Document hooks, restrictions (do not call from constructors), and what subclasses must preserve. Test by writing real subclasses before release. If you cannot or will not do that, **prohibit** extension with `final`, `sealed` or a private constructor and offer composition and interfaces instead.

S4. For overriding methods, LSP requires: **preconditions** may be weakened (accept more) but never strengthened; **postconditions** may be strengthened (promise more) but never weakened; **invariants** must hold; the **history constraint** forbids a subtype from making visibly changeable what the parent made unchangeable; and no new unexpected exceptions. Java's type system enforces part of it: parameter types must match exactly (otherwise it is an overload), return types are covariant, checked exceptions only narrow, access only widens. Behavior is not checked. Test it with an **abstract contract test**: write the tests once against the parent type and run every implementation through them.

```
abstract class ListContractTest {
    abstract List<String> create();                          // each
implementation supplies an instance
    @Test void addThenGet()   { List<String> l = create(); l.add("a");
assertEquals("a", l.get(0)); }
    @Test void indexOutOfRange() {
assertThrows(IndexOutOfBoundsException.class, () -> create().get(0)); }
}
class ArrayListTest  extends ListContractTest { List<String> create() {
return new ArrayList<>(); } }
class LinkedListTest extends ListContractTest { List<String> create() {
return new LinkedList<>(); } }
```

S5. *Subclassing* `PaymentRepository` is fragile: it needs a non-final class, ties you to its internals, only wraps one class, and DI containers must instantiate your subclass. A **Decorator** (`LoggingPaymentRepository implements PaymentRepository` that delegates to the real one) is explicit, easy to test and works with any implementation, at the cost of delegating every method by hand. **Spring AOP** (`@Around` advice or Micrometer's `@Timed`) applies the concern declaratively to many beans at once, but works through proxies, so it has limits: self-invocation bypasses it, `final` and `private` methods are not intercepted, and it is runtime magic that is harder to debug. Choose AOP for cross-cutting concerns that apply to many beans (logging, timing, transactions), a Decorator for one targeted concern with a clear interface, and avoid subclassing.

S6. For a class with no interface, Spring (with CGLIB, and by default in Spring Boot) generates a **runtime subclass** of the bean class and registers an instance of it. That subclass **overrides** each public, protected or package-private non-final method to run the interceptor chain (open a transaction, call the real method on the target, commit or roll back). **Final class:** it cannot be subclassed, so the proxy cannot be created and startup fails (or, for some setups, the advice is skipped). **Final method:** it cannot be overridden, so a call on the proxy runs the method directly with no interceptor: no transaction, with no error. **Private method:** not overridable, so it is not intercepted either. The same bypass happens for **self-invocation** (`this.save()` inside the bean calls the target directly, not the proxy) and for static methods. Fixes: remove `final` , make the annotated method non-private, route the call through another bean or an injected self-reference, or use AspectJ weaving. (Kotlin classes are final by default, which is why the Spring Kotlin plugin opens them.)

S7. The diamond problem for defaults: a class inherits the same method signature from two interfaces. The rules: (1) **A concrete or abstract method declared in a class or its superclass wins** over any interface default. (2) Otherwise the **most specific** interface wins, meaning one that extends the other and overrides the default. (3) Otherwise the compiler reports an error and the class must **override** the method, optionally delegating with `X.super.m()` (where `X` is a direct superinterface). If an abstract class declares an abstract `hello()` and an interface supplies a default `hello()`, rule 1 applies: the abstract class's declaration wins, so the concrete subclass **must implement** it, and the default is used only if the implementation calls `Iface.super.hello()`.

S8. A **bridge method** is a synthetic method the compiler generates so that overriding still works at the JVM level, where methods match by exact descriptor. For a covariant return, `Dog` gets a hidden `Animal reproduce()` that calls the real `Dog reproduce()`. For generics:

```
class Name implements Comparable<Name> {
    public int compareTo(Name o) { ... }                    // what you
wrote
    // synthetic, generated by javac (ACC_BRIDGE | ACC_SYNTHETIC):
    // public int compareTo(Object o) { return compareTo((Name) o); }
}
```

The interface's erased method is `compareTo(Object)`, so the bridge implements that and casts. You see it with `javap -p -c Name`, in stack traces, and in reflection (`Method.isBridge()`). A call through a raw `Comparable` with the wrong type fails with `ClassCastException` inside the bridge.

S9. Use an **identity map** from original to copy to handle cycles and shared parts: before copying an object, look it up (`IdentityHashMap<Object, Object>`); if present, reuse the existing copy; otherwise create the copy, register it **before** copying its fields (so a back-reference finds it), then copy the fields recursively.

```
Object copy(Object o, Map<Object, Object> seen) {
    if (o == null || isImmutable(o)) return o;               // Strings,
boxed numbers, enums, records of immutables
    Object existing = seen.get(o);
    if (existing != null) return existing;                    // shared node
or cycle: reuse
    Object c = newInstanceLike(o);
    seen.put(o, c);                                           // register
BEFORE recursing
    for (Field f : mutableFieldsOf(o)) set(c, f, copy(get(o, f), seen));
    return c;
}
```

This is what serialization does internally. Long object chains can overflow the stack, so use an explicit work queue for very deep graphs. Avoid deep copying when you can: make types **immutable**, use persistent data structures with **structural sharing**, copy-on-write, or map entities to DTOs (JPA) so you never copy a live object graph.

S10. If `Parent.clone()` is written as `return new Parent(this);` instead of `super.clone()`, the result is always a `Parent`. A subclass `Child.clone()` calls `super.clone()` and casts to `Child`: **`ClassCastException`**, because the object returned is not a `Child`. The rule is that a `clone()` must return `super.clone()`'s result so the runtime class is preserved. Bloch's guidelines: implement `Cloneable` and a **public** `clone()` with a covariant return type and no checked exception; obtain the object with `super.clone()` and never with a constructor; **fix up** mutable fields (copy arrays and collections, deep-copy elements if needed); do not call overridable methods inside `clone()`; do not make `final` fields mutable just to clone. Better: a **copy constructor or copy factory**, and `clone()` only for arrays.

S11. A virtual call costs a table lookup, which is small, and the JIT removes most of it for monomorphic and bimorphic sites; megamorphic hot paths cost more because they are not inlined. Benchmark with JMH before optimizing. The deeper trade-off is the **expression problem**. With **virtual dispatch**, adding a **new type** is easy (write a class that implements the interface) but adding a **new operation** means editing every class. With a **sealed hierarchy plus a pattern-matching `switch`**, adding a **new operation** is easy (write another function) and adding a **new type** makes the compiler flag every non-exhaustive `switch`, which is safe but touches many places. Use sealed plus `switch` for closed domain models (ASTs, events, results), and interfaces for open, plugin-style extension.

S12. A `protected` field is part of the contract with **every subclass, forever**. Any subclass can corrupt the invariant (`balance` negative), you cannot change the representation, you cannot add validation or logging, and visibility across threads is unmanaged. Same-named fields in subclasses hide each other and confuse readers. Redesign: make the field `private`, offer operations that enforce rules, and expose narrow hooks.

```
class Account {
    private double balance;                                    // nobody else
can touch it
    public double getBalance() { return balance; }
    public final void debit(double amt) {                      // invariant
enforced in one place
        if (amt <= 0 || amt > balance) throw new IllegalArgumentException();
        balance -= amt;
        onDebit(amt);
    }
    protected void onDebit(double amt) { }                     // a narrow,
documented hook
}
class AuditedAccount extends Account {
    @Override protected void onDebit(double amt) { audit("debit " + amt); }
}
```

End of Part 3.3. Next is **Part 3.4: Interfaces in depth** (members and their rules, `extends` vs `implements`, default, static and private methods, naming conflicts, marker, functional and sealed interfaces, abstract class vs interface). Tell me when to start, or flag anything here you want expanded first.